Ders 9 Genişletilmiş Notu: Vize Değerlendirmesi, Özyinelemeli Algoritmalar ve Rekürans Bağıntılarının Çözümü

Ders 9 Genişletilmiş Notu: Vize Değerlendirmesi, Özyinelemeli Algoritmalar ve Rekürans Bağıntılarının Çözümü

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Vize Sorularında En Çok Yapılan Hatalar
- 2. Yinelemeli ve Özyinelemeli Algoritmalar
- 3. Gauss'un Toplamı ve Rekürsif Toplama
- 4. Faktöriyelin Rekürsif Hesabı ve Yürütme Zamanı
- 5. Rekürans Bağıntısı Nedir?
- 6. Rekürans Bağıntılarının Sınıflandırılması
- 7. Sabit Katsayılı Homojen Bağıntılar: Karakteristik Kök Yöntemi
- 8. Sabit Katsayılı Homojen Olmayan Bağıntılar
- 9. Sözel Problemlerden ve Sayı Dizilerinden Rekürans Bağıntısı
- 10. Çözülebilir, Çözülemez ve Çözümsüz Problemler

1. Vize Sorularında En Çok Yapılan Hatalar

Ders, vizeden sonraki ilk dersti. Hoca yeni konuya geçmeden önce yaklaşık yarım saat boyunca vize sorularında sınıfın çoğunlukla nerede hata yaptığını anlattı. Amacı, aynı hataların finalde tekrar edilmemesiydi.

1.1 Bağıntı Sorusu

Birinci soruda bir bağıntı verilmiş ve bağıntının yansıyan (reflexive) olup olmadığı gibi özellikleri sorulmuştu. Sınıfın büyük çoğunluğu bu soruyu doğru yapmıştı. Hatalar, bağıntıyı baştan yanlış yazan öğrencilerdeydi. Bağıntı yanlış yazılınca diğer şıklar da otomatik olarak o yanlış bağıntıya göre çözülüyor ve sorudan puan alınamıyor.

1.2 Prefix ve Postfix Sorusu

Hoca bu soruda en büyük hatanın kökün yazılışında olduğunu söyledi. İfadenin başı herkeste doğruydu. Ancak iki hata sık yapılmıştı.

Birinci hata: örtük çarpma. İfadede bir parantez ve hemen yanında başka bir terim vardı. İkisinin arasında aslında bir çarpma işlemi vardır. Toplama ve çıkarmayı işaret olarak yazarız, ama çarpmayı genelde yazmayız. $(A + B)C$ gördüğümüzde bunu "A ile B'yi topla, C ile çarp" diye okuruz. Prefix veya postfix'e çevirirken bu çarpma işareti mutlaka konmalıdır, yoksa işlem oluşmaz. Hoca bu hatanın az bir kesimde olduğunu söyledi.

İkinci hata: kökü üs olarak yazmamak. Asıl problem buydu. İfadede $\sqrt[n]{x}$ biçiminde bir kök vardı. Bu aslında

$$\sqrt[n]{x} = x^{1/n}$$

demektir. Bazı öğrenciler prefix'i "$x\,(1/n)$" gibi parantezli bir yazımla bırakmıştı. Böyle bir yazım olmaz. $1/n$ ifadesinin de prefix'e çevrilmesi gerekir: $/\,1\,n$. Yani önce 1 n'e bölünür, çıkan sonuç x'in kuvveti olarak alınır:

$$\sqrt[n]{x} \;\Rightarrow\uparrow\; x\;/\;1\;n$$

Hoca bir örnek daha verdi: $\sqrt[3]{x^2}$. Bu ifade $x^{2/3}$'tür. Önce $2/3$'ün prefix'i yazılır ($/\,2\,3$), sonra bu sonuç x'in kuvveti olarak alınır:

$$\sqrt[3]{x^2} = x^{2/3} \;\Rightarrow\uparrow\; x\;/\;2\;3$$

Postfix ifadenin değeri. Sorunun devamında postfix ifadelerin $n = 2$, $m = 1$ için değeri istenmişti. Sonuçlar 25 ve 432'ydi. 25'i çoğu öğrenci doğru bulmuştu. İkinci ifadede ise yanlış yapanların çoğu 532 bulmuştu. Hoca, bir hata yapıldığında herkesin aynı hatayı yapmayacağını, yanlışın kişiye özgü olacağını söyledi.

Sorunun bir şıkkında prefix ifadelerden birini yığın (stack) kullanarak değerlendirmek isteniyordu. Büyük çoğunluk bu kısmı yazmamış ya da hatalı yapmıştı.

Arada bir öğrenci ikinci ifadenin 432'ye nasıl çıktığını sordu. Hoca ekranı paylaşıp hesabı öğrenciyle birlikte yaptı. Bu çözüm "Örnek Sorular" bölümünde adım adım verildi.

Soru: Prefix ve postfix'te üslü ifadeler ters mi yazılıyor? **Cevap:** Hayır. 3^2'yi prefix yazarken işaret önde olur, sonra tabanı, sonra üssü yazarız: $\uparrow\,3\,2$. Bunu infix'e çevirirsek $3^2 = 9$ olur. Postfix'te de sıra aynıdır, işaret sona gelir: $3\,2\,\uparrow$, yine $3^2 = 9$. Sayıların yeri değiştirilirse, yani $\uparrow\,2\,3$ veya $2\,3\,\uparrow$ yazılırsa ifade $2^3 = 8$ olur. Yani iki yazım farklı sonuç verir.

1.3 Karnaugh Sorusu

Hoca bu soruda "şok geçirdiğini" söyledi. Soruda dört giriş vardı: kara yolu, demir yolu, deniz yolu ve hava yolu. Çıkış tekti: F fonksiyonu. Bir noktadan bir noktaya hangi yollar tercih edilerek gidiliyorsa doğruluk tablosunda o satıra 1, diğerlerine 0 konur. Örneğin "sadece kara yoluyla gider" deniyorsa, doğruluk tablosunda yalnız kara yolunun 1 olduğu satır 1 olmalıdır. Soruda altı şart verilmişti, dolayısıyla fonksiyon altı minterimden oluşuyordu:

$$F = \Sigma(4, 8, 9, 10, 12, 14)$$

Bazı öğrenciler sonuç doğru olsa da bu minterimleri bulamamıştı. Sebebi, Karnaugh haritasını 90 derece döndürerek yerleştirmeleriydi. O zaman minterimler farklı çıkıyor.

Gruplama sırası şöyledir: Önce 16'lık grup var mı diye bakılır, yok. 8'lik var mı, yok. 4'lük var mı, evet: iki 2'li grup katlanınca bir 4'lük oluşur. Başka 4'lük yok. Sonra 2'likler aranır. İki tane 2'lik vardır. Hoca bazı öğrencilerin bir 1'i komşusuyla 2'li grup yapmak mümkünken tek başına aldığını söyledi. Tek bir 1'i tek başına almak doğru sadeleştirme değildir.

Değişken sırası A, B, C, D ve A en anlamlı bit alınırsa (ara adım eklendi):

- 4'lük grup: $8, 10, 12, 14 \Rightarrow A\overline{D}$
- 2'lik grup: $8, 9 \Rightarrow A\overline{B}\,\overline{C}$
- 2'lik grup: $4, 12 \Rightarrow B\overline{C}\,\overline{D}$

$$F = A\overline{D} + A\overline{B}\,\overline{C} + B\overline{C}\,\overline{D}$$

Ders sonrası not: Derste harita ve gruplar sözlü anlatıldı; değişken adları verilmedi. Yukarıdaki A, B, C, D sırası varsayımdır. Python ile bütün asal implikant seçenekleri denendi: Bu üç terimli çözüm tektir ve en kısadır.

Sorunun “Karnaugh’ta bulduğunuz sonucu Bool cebriyle de ispatlayın” şıkkı iki grupta da iptal edildi. Hoca, sınavdan sonra çözümleri yaparken bu şıkkın çözümünün çok uzun sürdüğünü söyledi.

1.4 Karmaşıklık Sorusu

Hoca bu soruyu okurken “çok korktuğunu” söyledi: Büyük çoğunluk $O(n^3)$ bulmuştu. Oysa sonuç n^3 çıkamaz. Kodda iç içe üç döngü vardı:

```
for (i = n/2; i <= n; i++)
    for (j = 1; j + n/2 <= n; j++)
        for (k = 1; k <= n; k = k*2)
            ...
```

Dış döngü: i, $n/2$’den başlayıp n’e kadar gidiyor. Bu yaklaşık $n/2$ kez döner. Hoca ± 1 farkların önemli olmadığını, n’e bağlı ifadeyi aradığımızı söyledi. Sayısal örnek: $n = 10$ ise i 5’ten başlar, 10’a kadar gider.

Orta döngü: Hoca, bir şeyi çözemiyorsak hemen bir sayı koyup denemeyi önerdi. $n = 10$ olsun. $j = 1$ iken $j + n/2 = 6 \leq 10$. $j = 2$ iken 7, $j = 3$ iken 8, $j = 4$ iken 9, $j = 5$ iken $10 \leq 10$. Toplam 5 kez, yani yine $n/2$ kez döner.

İç döngü: Hoca burasının en önemli kısım olduğunu söyledi. $k = 1$ ile başlar ve her adımda 2 ile çarpılır. $n = 10$ için k: $1, 2, 4, 8$ olur, 16’da koşul bozulur.

> **Hocanın vurgusu:** Bir değişken her adımda 2 ile çarpılıyor ya da 2’ye bölünüyorsa, yani bir şey sürekli yarısı yarısı diye gidiyorsa, o döngünün karmaşıklığı $\log n$’dir. İkili aramayı (binary search) hatırlayın: Dizi n, sonra $n/2$, $n/4$, $n/8$, … diye 1’e kadar yarıya bölünür ve karmaşıklığı $\log n$’dir. 2 ile çarpmak da 2’ye bölmek de aynı sonucu verir.

Toplam: Üç döngünün tekrar sayıları çarpılır:

$$\frac{n}{2} \cdot \frac{n}{2} \cdot \log_2 n = \frac{n^2}{4} \log_2 n$$

Karmaşıklıkta en büyük terim alınır. $n/2$, n^2 ve $n^2 \log n$ karşılaştırılınca en büyüğü $n^2 \log n$’dir. Sonuç:

$$O(n^2 \log n)$$

> **Ders sonrası not:** Tam sayımda dış döngü $n/2 + 1$ kez, iç döngü $\lfloor \log_2 n \rfloor + 1$ kez döner. Python ile $n = 10, 100, 1000$ için en içteki gövdenin çalışma sayısı sayıldı (120, 17850, 2505000); bu değerler tam sayım formülüyle aynıdır. Sonuç $\Theta(n^2 \log n)$’dir, değişmez.

1.5 Önerme Sorusu

Bu soru kolaydı; herkes (a) şıkkını doğru yapmıştı. (b) şıkkında “ancak ve ancak finale girerseniz dersi geçebilirsiniz” cevabı kabul edildi. Bazı öğrenciler bu ifadeyi dışlayıcı veya (XOR) gibi yazmıştı. Bu yanlıştır. “Ancak ve ancak” ifadesinin sembolü çift yönlü oktur: $p \leftrightarrow q$.

2. Yinelemeli ve Özyinelemeli Algoritmalar

Hoca yeni konuya, yani rekürans bağıntılarına geçmeden önce, önceki derslerde yetişmediği için anlatamadığı özyinelemeli (recursive) fonksiyonları anlattı. Rekürans bağıntıları da aslında rekürsiftir, yani özyinelemeli fonksiyonlara girer. Hoca bu bölümün slaytlarını başka bir hocanın ders videosundan aldığını ve o anlatımı çok beğendiğini söyledi.

2.1 Yinelemeli Algoritmalar

Şimdiye kadar hep yinelemeli (iterative) algoritmaları gördük. Bunlarda bir giriş boyutu belirlenir, temel işlemler sayılır, en iyi, en kötü ve ortalama durumlara bakılır. Genelde en kötü durum (worst case) için n'e bağlı yürütme zamanı (running time) hesaplanır. Buradan da asimptotik gösterimle karmaşıklık çıkarılır.

2.2 Özyinelemeli Algoritmalar ve Başlangıç Koşulu

Özyinelemeli algoritma kendi kendini çağırır. Farkı şudur: Temel işlemin kaç kez yürütüldüğünü ifade etmek için uygun bir başlangıç koşulu (initial condition) gerekir. Bir problemi rekürsif çözüyorsak çözümde mutlaka bir başlangıç, yani durma koşulu olmalıdır.

Çalışma mantığı şöyledir: Program durma noktasına kadar ilerler ve her adımda elindeki veriyi yığına (stack) atar. Durma noktasına gelince yığından geri çeke çeke başlangıç durumuna döner ve sonucu bulur.

Hoca bunu bir $T(n)$ modeliyle anlattı. $T(n)$, hep bir önceki durumuna bağlıdır. $n = 5$ ise "git önce 4'ü çöz" der; 4 için "git önce 3'ü çöz", sonra 2, sonra 1. Başlangıç durumu olarak $T(1)$ verilmişse 1'e kadar gidilir, sonra geriye doğru işlem yapılarak $T(n)$ bulunur.

2.3 İleri Yerine Koyma

Rekürsif fonksiyonların birçok çözüm yöntemi vardır. Derste ileri yerine koyma (forward substitution) yöntemi referans alındı. Örnek bağıntı:

$$T(n) = T(n-1) + 1, \qquad T(1) = 2$$

Burada $n = 1$ durma noktasıdır ve değeri 2'dir. Başlangıç koşulundan ileri doğru hesaplanır:

- $T(2) = T(1) + 1 = 2 + 1 = 3$
- $T(3) = T(2) + 1 = 3 + 1 = 4$
- ...

Buradan genel çözüm tahmin edilir:

$$T(n) = n + 1$$

Rekürsif program yazıldığında ise hesap tersten başlar. $T(6)$'yı hesaplamak için $T(5)$ gerekir, o henüz hesaplanmamıştır; $T(5)$ için $T(4)$ gerekir, ..., $T(1)$'e kadar inilir. $T(1)$ verilmiştir. Sonra geriye doğru her bulunan değer yerine konarak sonuca ulaşılır.

Hoca çözümün doğruluğunu da gösterdi: $T(n) = n+1$ ise $T(n-1) = n$ olur. Bağıntıda yerine konunca $T(n-1) + 1 = n + 1$ çıkar; artı bir ve eksi bir birbirini götürür.

3. Gauss'un Toplamı ve Rekürsif Toplama

3.1 Gauss'un Hikâyesi

Hoca, matematik kitaplarında da geçen bir hikâye anlattı. Ünlü matematikçi Gauss çocukken biraz haşarıymış. Öğretmeni onu oyalamak için 1'den n'e kadar, örneğin 500'e kadar olan sayıları toplamasını istemiş. Gauss birkaç dakika sonra cevabı vermiş, öğretmen de şaşırmış.

Gauss sayıları önce düz, sonra altına tersten yazmış:

$$\begin{array}{ccccccc} 1 & + & 2 & + & \cdots & + & n \\ n & + & (n-1) & + & \cdots & + & 1 \end{array}$$

Her sütunun toplamı $n + 1$'dir. Örneğin $2 + (n-1) = n + 1$, $3 + (n-2) = n + 1$. Toplam n sütun vardır. Her sayı iki kez toplandığı için sonuç ikiye bölünür:

$$1 + 2 + \cdots + n = \frac{n(n+1)}{2}$$

$n = 500$ için sonuç $\frac{500 \cdot 501}{2} = 125\,250$'dir (ara adım eklendi; Python ile doğrulandı).

3.2 Toplamı Rekürsif Yazmak

Hoca aynı işlemin rekürsif yapılıp yapılamayacağını sordu. 1'den n'e kadar toplamı veren fonksiyon:

$$f(n) = n + f(n-1), \qquad f(1) = 1$$

$f(5)$ hesaplanırken:

- $f(5) = 5 + f(4)$, $f(4)$ bilinmiyor.
- $f(4) = 4 + f(3)$, $f(3)$ bilinmiyor.
- $f(3) = 3 + f(2)$, $f(2)$ bilinmiyor.
- $f(2) = 2 + f(1)$, $f(1) = 1$ başlangıç koşulu olarak verilmiş.

Geri dönerken: $f(2) = 2 + 1 = 3$, $f(3) = 3 + 3 = 6$, $f(4) = 4 + 6 = 10$, $f(5) = 5 + 10 = 15$.

Yığın açısından: 5 yığına atılır, sonra 4, 3, 2. $f(1)$ eldedir. Sonra yığından 2 çekilir, toplanır, 3 olur; 3 çekilir, 6 olur; 4 çekilir, 10 olur; 5 çekilir, 15 olur. Yığın biter ve sonuç döner.

> **Hocanın vurgusu:** Başlangıç koşulu rekürsif fonksiyonlarda kesinlikle verilmelidir.

Hoca bu tip bağıntıların aslında rekürans bağıntıları olduğunu söyledi. Matematiksel olarak $a_n = n + a_{n-1}$ diye de yazılabilir. Ancak kodlarken fonksiyon biçiminde ($f(n)$) göstermek daha kolaydır. Burada günlük hayattan bir problem ("1'den n'e kadar topla") rekürans bağıntısına çevrilmiş olur.

4. Faktöriyelin Rekürsif Hesabı ve Yürütme Zamanı

Faktöriyel yinelemeli bir algoritmayla da bulunabilir, ama derste rekürsif yazıldı:

```
factorial(n):
    if n == 0
        return 1
    return n * factorial(n - 1)
```

n sıfırsa faktöriyel 1'dir, çünkü $0! = 1$; başka işlem yapılmaz.

Örnek: $n = 4$**.** $4! = 4 \cdot 3 \cdot 2 \cdot 1$ olmalıdır. Fonksiyon $4\cdot$ factorial(3)'ü çağırır. $3 \neq 0$ olduğu için $3\cdot$ factorial(2) çağrılır, sonra $2\cdot$ factorial(1), sonra $1\cdot$ factorial(0). Durma noktasında 1 döner. Yığından geri çekilirken $1 \to 2 \to 6 \to 24$ bulunur. Dönen değer 24'tür.

Temel işlemlerin sayılması. $n = 0$ ise yalnız bir karşılaştırma yapılır. $n \neq 0$ ise her çağrıda bir karşılaştırma, bir çarpma ve bir çıkarma ($n - 1$) yapılır, yani 3 işlem. Buradan yürütme zamanı bağıntısı:

$$T(n) = T(n-1) + 3, \qquad T(0) = 1$$

Geriye doğru yerine koyma. $T(n-1) = T(n-2) + 3$ yerine konur:

$$T(n) = T(n-2) + 6 = T(n-3) + 9 = \cdots = T(n-k) + 3k$$

Durma noktası $T(0) = 1$'dir. $n - k = 0$ için $k = n$ olur:

$$T(n) = T(0) + 3n = 3n + 1$$

Karmaşıklık $O(n)$'dir. Python ile ilk değerler $1, 4, 7, 10, 13, \ldots$ olarak kontrol edildi.

> **Ders sonrası not:** Hoca işlemlerin "$n-1$ kez" tekrarlandığını söyledi. Rekürsif çağrı $n, n-1, \ldots, 1$ için, yani n kez çarpma ve çıkarma yapar. $T(n) = 3n+1$ sonucu zaten n tekrara göredir; karmaşıklık değişmez.

Hoca internette "recursive functions" diye aranırsa Fibonacci sayılarının ve hızlı sıralamanın (quick sort) rekürsif çözümü gibi birçok örnek bulunabileceğini söyledi.

Rekürsif mi, yinelemeli mi? Yinelemeli yazmak daha kolaydır. İnsan durup dururken programı rekürsif yazmaz. Ancak problem matruşka bebekleri gibiyse, yani bir bebeği açınca içinden bir bebek, onu açınca yine bir bebek çıkıyorsa, sürekli aynı işlemi yaparak ilerleyen bir yapı varsa rekürsif yazılabilir. Rekürsif program yazmak zahmetlidir.

5. Rekürans Bağıntısı Nedir?

Hoca bu konunun biraz uzun olduğunu ve çok iyi dinlenmesi gerektiğini söyledi. İngilizcesi "recurrence"tır. Türkçede "rekürans" diye çeviren kitaplar vardır, ama Türkçede böyle bir kelime yoktur. Anlamı "kendini tekrar eden"dir; Türkçede öz yineli denir.

> **Hocanın vurgusu:** Bir dizinin içinde kendisinden bir parça bulunuyorsa, o diziye rekürans bağıntısı (öz yineli dizi) denir.

5.1 Dizi ile Farkı

Önce bildiğimiz bir dizi (sequence) düşünelim:

$$a_n = 5n + 1$$

Diziyi hesaplamak kolaydır. 20. elemanı bulmak için n yerine 20 konur: $a_{20} = 5 \cdot 20 + 1 = 101$. Buradaki a_n ve $5n$ ikisi de yalnız n'e bağlıdır; içinde dizinin kendisinden bir parça yoktur.

Rekürans bağıntısında ise sağ tarafta a_{n-1} gibi, dizinin kendisinden türetilmiş bir parça vardır. Bu yüzden dizi gibi hesaplanamaz. a_{20} istenirse n yerine 20 konunca sağ tarafta a_{19} çıkar. a_{19}'un değerini bilmeden işlem yapılamaz.

Bir bağıntıda kendisinden türetilmemiş terimler de olabilir (örneğin yalnız n'e bağlı bir terim). Bu, onun rekürans bağıntısı olmasını engellemez. İçinde en az bir tane kendisinden türetilmiş terim olması yeterlidir.

Hoca, $n \geq 1$ için tanımlı birinci dereceden bir bağıntıda a_1 hesaplanırken sağ tarafta a_0 çıkacağını söyledi. Bu yüzden a_0 başlangıç değerine ihtiyaç vardır. Rekürans bağıntılarında başlangıç değerleri mutlaka verilmelidir.

5.2 Örnek: a_5'in Hesabı

Verilen bağıntı:

$$a_n = 2a_{n-1} - a_{n-2} + 2^{n-1}, \qquad a_0 = 1,\ a_1 = 2,\ n \geq 2$$

a_5 isteniyor. Bunun için a_4, a_3, a_2, a_1 bilinmelidir. Neden a_0 ve a_1 verilmiş? Bağıntının derecesi 2'dir. $n = 1$ alınsaydı sağ tarafta a_0 ve a_{-1} çıkardı; a_{-1} diye bir şey yoktur. Demek ki $n = 1$ olamaz. $n = 2$ için a_1 ve a_0 gerekir. Bu yüzden iki başlangıç değeri verilmiş ve $n \geq 2$ denmiştir.

n	Hesap	a_n
2	$2a_1 - a_0 + 2^1 = 2 \cdot 2 - 1 + 2$	5
3	$2a_2 - a_1 + 2^2 = 2 \cdot 5 - 2 + 4$	12
4	$2a_3 - a_2 + 2^3 = 2 \cdot 12 - 5 + 8$	27
5	$2a_4 - a_3 + 2^4 = 2 \cdot 27 - 12 + 16$	58

Derste $a_2 = 5$, $a_3 = 12$, $a_4 = 27$ söylendi; a_5'in değeri sözlü verilmedi. $a_5 = 58$ (ara adım eklendi; Python ile doğrulandı).

Dizilerde n yerine 5 konup sonuç doğrudan bulunur. Burada ise adım adım, bir önceki durumu kullana kullana ilerlenir.

5.3 Bu Konu Neden Ayrık Matematikte?

Hoca iki amaç saydı:

- **Verilen bir rekürans bağıntısını çözmek.** Bağıntılar doğrusal (linear) veya doğrusal olmayan (non-linear) olabilir; ikisi çok farklı çözülür.
- **Sözel bir problemi rekürans bağıntısına çevirip çözmek.** Faktöriyel ve 1'den n'e kadar toplam da aslında sözel problemlerdi. Hoca bunun en zor kısım olduğunu, bazı problemlerin bağıntıya çevrilmesinin çok zor olabileceğini ve öyle problemlerin dersin kapsamında olmadığını söyledi.

6. Rekürans Bağıntılarının Sınıflandırılması

6.1 Derece (Order)

Kendisinden türetilmiş terimler arasında geriye kayması en büyük olan terim, bağıntının derecesini verir.

- Yalnız a_{n-1} varsa birinci derecedendir.
- a_{n-1} ve a_{n-2} varsa ikinci derecedendir.
- a_{n-1} ve a_{n-3} varsa (yani a_{n-2}'nin katsayısı 0) üçüncü derecedendir.
- Yalnız a_{n-4} varsa dördüncü derecedendir.

6.2 Homojen ve Homojen Olmayan

Bağıntı yalnız kendisinden türetilmiş terimlerden oluşuyorsa homojendir (homogeneous). Bunların dışında başka bir terim varsa homojen değildir. Homojenliği bozan terimlere derste verilen örnekler: bir sabit (örneğin -4), $4n$, 3^n, n^2, n^3.

6.3 Doğrusal ve Doğrusal Olmayan

Bağıntıdaki rekürans terimlerinin kuvveti alınıyorsa (örneğin a_{n-1}^2) ya da terimler birbiriyle çarpılıyorsa (örneğin $a_{n-1}a_{n-2}$), bağıntı doğrusal değildir. Hoca bu kuralın yalnız sağ taraf için değil, sol taraf için de geçerli olduğunu söyledi: Sol tarafta a_n^3 varsa bağıntı yine doğrusal değildir.

> **Ders sonrası not:** Hoca bir an doğrusallığı bozan terimlerin "homojen olup olmamayla alakalı" olduğunu söyledi, sonra "bunu unutun" diyerek düzeltti. Doğrusallık terimlerin kuvveti ve çarpımıyla, homojenlik ise rekürans terimi olmayan ek terimlerle ilgilidir.

6.4 Sabit Katsayılı ve Sabit Katsayılı Olmayan

Rekürans terimlerinin önündeki katsayılar sabit sayıysa (3, 5, 7 gibi) bağıntı sabit katsayılıdır. Katsayıda n'e bağlı bir çarpan varsa, örneğin $n\,a_{n-1}$ veya $3^n a_{n-2}$, sabit katsayılı değildir.

6.5 Özet Tablosu

Özellik	Anlamı	Bozan durum
Derece	En büyük geriye kayma (a_{n-k}'daki k)	—
Homojen	Yalnız kendisinden türetilmiş terimler	Sabit, $4n$, 3^n, n^2 gibi terimler
Doğrusal	Terimler birinci kuvvette, birbiriyle çarpılmamış	a_{n-1}^2, $a_{n-1}a_{n-2}$, a_n^3
Sabit katsayılı	Katsayılar sabit sayı	$n\,a_{n-1}$, $3^n a_{n-2}$

7. Sabit Katsayılı Homojen Bağıntılar: Karakteristik Kök Yöntemi

Bağıntı sabit katsayılı ve homojense, çözümde karakteristik kök (characteristic root) yöntemi kullanılır. Bu yöntem ikinci ve daha yüksek dereceli bağıntılar için kullanılır. Hoca birinci dereceden bağıntıların farklı çözüldüğünü ve gerçek dünyada pek karşımıza çıkmadığını söyledi. Homojen olmayan bağıntılarda da homojen kısım yine bu yöntemle çözülür.

Homojen bağıntının çözümü $a_n^{(h)}$ ile gösterilir; h homojenin baş harfidir. Homojen olmayan bir bağıntıda buna bir de özel çözüm $a_n^{(\ddot{o})}$ eklenir (bkz. Bölüm 8).

7.1 Karakteristik Denklemin Kurulması

Hocanın örneği:

$$a_n = 3a_{n-1} + 4a_{n-2}$$

Katsayılar sabit (3 ve 4), bağıntı homojen ve ikinci derecedendir. Karakteristik denklem kurulurken en düşük indisli terime 1, bir üstüne r, bir üstüne r^2 karşılık getirilir:

- $a_{n-2} \to 1$
- $a_{n-1} \to r$
- $a_n \to r^2$

a_{n-3} diye bir terim daha olsaydı sıra $1, r, r^2, r^3$ olurdu. Denklem:

$$r^2 = 3r + 4 \;\Rightarrow\; r^2 - 3r - 4 = 0 \;\Rightarrow\; (r-4)(r+1) = 0$$

Kökler $r_1 = 4$ ve $r_2 = -1$'dir.

> **Ders sonrası not:** Hoca kökleri bulurken "çarpımları 4, toplamları -3" dedi. $r^2 - 3r - 4 = 0$ için köklerin çarpımı -4, toplamı 3'tür. Kökler yine 4 ve -1'dir.
>
> **Ek bilgi (derste anlatılmadı):** Karakteristik denklem, çözümün $a_n = r^n$ biçiminde olduğu varsayılarak elde edilir. $r^n = 3r^{n-1} + 4r^{n-2}$ eşitliği r^{n-2} ile bölününce $r^2 = 3r + 4$ kalır. Bu yüzden her kök bir çözüm verir. Bağıntı doğrusal olduğu için çözümlerin sabitlerle çarpılıp toplanması da çözümdür.

7.2 Ayrık ve Çakışık Kökler

İkinci dereceden denklemin iki durumu vardır:

- **İki farklı (ayrık) kök** $r_1 \neq r_2$:

$$a_n^{(h)} = C_1 r_1^n + C_2 r_2^n$$

- **Çakışık (eşit) iki kök** $r_1 = r_2 = r$: Tek fark, C_2'nin yanına bir n çarpanı eklenmesidir.

$$a_n^{(h)} = C_1 r^n + C_2\, n\, r^n$$

Örneğimizde kökler ayrıktır:

$$a_n^{(h)} = C_1\, 4^n + C_2\, (-1)^n$$

Ancak bu henüz tam çözüm değildir; C_1 ve C_2 bilinmiyor. Hoca, a_5 sorulduğunda n yerine 5 koyup doğrudan sonuç bulabilmek için C_1 ve C_2'nin de hesaplanması gerektiğini söyledi.

7.3 Üçüncü ve Daha Yüksek Derece

- **Bütün kökler farklıysa:** $C_1 r_1^n + C_2 r_2^n + C_3 r_3^n$. Dördüncü derecede bir $C_4 r_4^n$ daha eklenir.
- **Bütün kökler çakışıksa (r):** İkinci terimden itibaren n'in kuvvetleri eklenir: $C_1 r^n + C_2\, n\, r^n + C_3\, n^2 r^n$. Dördüncüsü olsaydı $C_4\, n^3 r^n$ olurdu.
- **İki kök çakışık, biri farklıysa:** Çakışık olanlar önce yazılır, ikincisine n çarpanı konur; çakışık olmayan kök standart biçimde eklenir: $C_1 r_1^n + C_2\, n\, r_1^n + C_3 r_3^n$.

Hoca üçüncü dereceden örnek yapmayacağını, çünkü mantığın aynı olduğunu söyledi.

> **Hocanın vurgusu:** Derece kaçsa o kadar başlangıç değeri verilmelidir. İkinci derecede a_0 ve a_1, birinci derecede a_0, üçüncü derecede a_0, a_1, a_2 verilmezse bağıntı çözülemez.

7.4 Örnek: Ayrık Kökler

$$a_n = 3a_{n-1} + 10a_{n-2}, \qquad a_0 = 5,\ a_1 = 8$$

Homojen, sabit katsayılı, ikinci dereceden. Karakteristik denklem:

$$r^2 = 3r + 10 \ \Rightarrow\ r^2 - 3r - 10 = 0 \ \Rightarrow\ (r-5)(r+2) = 0$$

Kökler 5 ve -2. Genel çözüm:

$$a_n = C_1\, 5^n + C_2\, (-2)^n$$

Başlangıç değerleri yerine konur:

- $n = 0$: $C_1 \cdot 5^0 + C_2 \cdot (-2)^0 = C_1 + C_2 = 5$
- $n = 1$: $5C_1 - 2C_2 = 8$

C_2'ler birbirini götürsün diye ilk denklem 2 ile çarpılıp ikinciyle toplanır: $2C_1 + 5C_1 = 10 + 8$, yani $7C_1 = 18$, $C_1 = \frac{18}{7}$. Yerine konunca $C_2 = 5 - \frac{18}{7} = \frac{17}{7}$.

$$a_n = \frac{18}{7}\, 5^n + \frac{17}{7}\, (-2)^n$$

Artık a_3 sorulursa n yerine 3 konup doğrudan bulunur. Python ile ilk 10 terim bağıntının kendisiyle karşılaştırıldı; formül doğrudur.

> **Ders sonrası not:** Hoca çarpanlara ayırırken önce "köklerim 5 ve 2" dedi. Kökler 5 ve -2'dir; çözümde doğru olarak -2 kullanıldı.

7.5 Örnek: Çakışık Kökler

$$a_n = -4a_{n-1} - 4a_{n-2}, \qquad a_0 = 3,\ a_1 = 5$$

Homojen, sabit katsayılı, derecesi 2. Karakteristik denklem:

$$r^2 + 4r + 4 = 0 \ \Rightarrow\ (r+2)^2 = 0$$

Kökler -2 ve -2, çakışık. İkinci terime n çarpanı eklenir:

$$a_n = C_1\, (-2)^n + C_2\, n\, (-2)^n$$

- $n = 0$: $C_1 = 3$
- $n = 1$: $-2C_1 - 2C_2 = 5 \Rightarrow -6 - 2C_2 = 5 \Rightarrow C_2 = -\frac{11}{2}$ (ara adım eklendi)

$$a_n = 3\,(-2)^n - \frac{11}{2}\,n\,(-2)^n$$

Python ile doğrulandı.

> **Ders sonrası not:** Hoca bir an "a_1 bir ise" dedi, aynı cümlede 5'e eşitledi. Doğru başlangıç değeri $a_1 = 5$'tir; $C_2 = -\frac{11}{2}$ bu değerle çıkar.

8. Sabit Katsayılı Homojen Olmayan Bağıntılar

Hoca problemlerin her zaman sabit katsayılı ve homojen gelmeyeceğini söyledi. Homojen olmayan bağıntıda rekürans terimlerinin dışında bir $f(n)$ terimi vardır. Çözüm iki parçalıdır:

$$a_n = a_n^{(h)} + a_n^{(\ddot{o})}$$

Önce homojen kısım (f(n) yokmuş gibi) karakteristik kök yöntemiyle çözülür. Sonra homojenliği bozan kısım için özel çözüm $a_n^{(\ddot{o})}$ bulunur. İkisi toplanır. C_1 ve C_2 en son, başlangıç değerleriyle bulunur.

$f(n)$ şunlar olabilir:

- **Polinom:** Sabit sayı (sıfırıncı dereceden polinom), $n + 2$ gibi birinci dereceden, n^2 gibi ikinci dereceden, n^3 gibi üçüncü dereceden.
- **Üstel fonksiyon:** $3 \cdot 2^n$, 5^n, 7^{-n} gibi.

> **Hocanın vurgusu:** Homojen kısmı yazdıktan sonra hemen C_1 ve C_2'yi çözmeye kalkmayın. Bağıntı yalnız homojen kısımdan oluşmuyor; önce özel çözüm eklenmeli.

8.1 $f(n)$ Sabit

$$a_n = 2a_{n-1} + 3a_{n-2} + 5, \qquad a_0 = 4,\ a_1 = 10$$

Homojen kısım: $r^2 = 2r + 3 \Rightarrow r^2 - 2r - 3 = 0 \Rightarrow (r-3)(r+1) = 0$. Kökler 3 ve -1, ayrık:

$$a_n^{(h)} = C_1\,3^n + C_2\,(-1)^n$$

Özel çözüm: Slayttaki not: "Polinomun derecesi ne ise a_n'nin derecesi de aynı olur." Yani $f(n)$ sabitse özel çözüm de bir sabittir. Bu sabitin ne olduğunu bilmediğimiz için A deriz. a_n, a_{n-1}, a_{n-2}'nin hepsi A'ya eşitlenir:

$$A = 2A + 3A + 5 \;\Rightarrow\; -4A = 5 \;\Rightarrow\; A = -\frac{5}{4}$$

Genel çözüm:

$$a_n = C_1\,3^n + C_2\,(-1)^n - \frac{5}{4}$$

Hoca C_1 ve C_2'nin başlangıç değerleriyle bulunacağını söyledi ama hesabı yapmadı: $n = 0$ için ifade 4'e, $n = 1$ için 10'a eşitlenir.

> **Çözüm (derste verilmedi):** $C_1 + C_2 - \frac{5}{4} = 4 \Rightarrow C_1 + C_2 = \frac{21}{4}$. $3C_1 - C_2 - \frac{5}{4} = 10 \Rightarrow 3C_1 - C_2 = \frac{45}{4}$. Toplanınca $4C_1 = \frac{66}{4}$, $C_1 = \frac{33}{8}$, $C_2 = \frac{9}{8}$. Sonuç: $a_n = \frac{33}{8}3^n + \frac{9}{8}(-1)^n - \frac{5}{4}$ (Python ile doğrulandı).

8.2 $f(n)$ Birinci Dereceden Polinom

$$a_n = a_{n-1} + 2a_{n-2} + 3n + 4, \qquad a_0 = 5,\ a_1 = 7$$

$f(n) = 3n + 4$ birinci dereceden bir polinomdur.

Homojen kısım: $r^2 = r + 2 \Rightarrow r^2 - r - 2 = 0 \Rightarrow (r-2)(r+1) = 0$. Kökler 2 ve -1, ayrık:

$$a_n^{(h)} = C_1\, 2^n + C_2\, (-1)^n$$

Özel çözüm: $f(n)$ sabitken özel çözüm sabit bir A idi. Birinci dereceden polinomsa özel çözüm de birinci dereceden bir polinomdur:

$$a_n^{(\ddot{o})} = An + B$$

a_{n-1} için n yerine $n-1$ konur, a_{n-2} için $n-2$:

- $a_{n-1} = A(n-1) + B = An - A + B$
- $a_{n-2} = A(n-2) + B = An - 2A + B$

Bağıntıda yerine konur:

$$An + B = (An - A + B) + 2(An - 2A + B) + 3n + 4 = 3An - 5A + 3B + 3n + 4$$

Terimler bir tarafa toplanınca:

$$-2An + 5A - 2B = 3n + 4$$

İki tarafın n'li terimleri ve sabitleri eşitlenir:

- n'li terimler: $-2A = 3 \Rightarrow A = -\frac{3}{2}$
- Sabitler: $5A - 2B = 4 \Rightarrow -\frac{15}{2} - 2B = 4 \Rightarrow B = -\frac{23}{4}$ (ara adım eklendi)

$$a_n^{(\ddot{o})} = -\frac{3}{2}n - \frac{23}{4}$$

Soru: Sondaki $-2A = 3$ adımı nasıl çıktı? **Cevap:** Sol tarafta $-2An + 5A - 2B$ var. Sağ taraftaki $3n$ soldaki n'li ifadeye, sağdaki 4 ise soldaki sabit $5A - 2B$'ye karşılık gelir. "n'ler gitti" denmesinin sebebi budur: n'li ifadeler eşitlenince $-2A = 3$ kalır ve buradan A çekilir.

Genel çözüm:

$$a_n = C_1\, 2^n + C_2\, (-1)^n - \frac{3}{2}n - \frac{23}{4}$$

$n = 0$ için 5'e, $n = 1$ için 7'ye eşitlenerek C_1 ve C_2 bulunur. Derste bu hesap yapılmadı.

> **Çözüm (derste verilmedi):** $C_1 + C_2 - \frac{23}{4} = 5 \Rightarrow C_1 + C_2 = \frac{43}{4}$. $2C_1 - C_2 - \frac{3}{2} - \frac{23}{4} = 7 \Rightarrow 2C_1 - C_2 = \frac{57}{4}$. Toplanınca $3C_1 = 25$, $C_1 = \frac{25}{3}$, $C_2 = \frac{29}{12}$ (Python ile doğrulandı).

8.3 $f(n)$ Üstel, Homojen Çözümde Geçmiyor

$$a_n = -4a_{n-1} - 3a_{n-2} + 2 \cdot 5^n, \qquad a_0 = 6,\ a_1 = 10$$

Homojen kısım: En küçük indisli terime 1, sonra r, r^2: $r^2 = -4r - 3 \Rightarrow r^2 + 4r + 3 = 0 \Rightarrow (r+3)(r+1) = 0$. Kökler -3 ve -1, ayrık:

$$a_n^{(h)} = C_1\,(-3)^n + C_2\,(-1)^n$$

Kontrol: $f(n)$ üstel olduğunda dikkat edilmesi gereken bir durum vardır. $f(n)$'deki üstel terim (5^n) homojen çözümde geçiyor mu? Köklerden biri 5 olsaydı homojen çözümde 5^n çıkardı. Bu örnekte kökler -3 ve -1'dir, 5^n geçmiyor.

> **Hocanın vurgusu:** Üstel terim homojen çözümdeki terimlerden birinde geçiyorsa özel çözüm n ile, iki terimde geçiyorsa n^2 ile, üç terimde geçiyorsa n^3 ile çarpılır. Hoca bu durumun öğrencilerin en çok atladığı nokta olduğunu ve genelde bunun sorulduğunu söyledi.

Özel çözüm: $f(n) = 2 \cdot 5^n$ olsa da özel çözümde 2 katsayısı kullanılmaz. Çünkü aradığımız şey, bağıntıya konunca a_n'yi sağlayan bilinmeyen bir katsayıdır. Sabit durumda A kullandığımız gibi burada da $A \cdot 5^n$ kullanılır:

$$a_n^{(\ddot{o})} = A \cdot 5^n$$

- $a_{n-1} = A \cdot 5^{n-1} = \frac{A}{5}\,5^n$
- $a_{n-2} = A \cdot 5^{n-2} = \frac{A}{25}\,5^n$

Hoca $5^{-1} = \frac{1}{5}$ olduğu için bu yazımın işi kolaylaştırdığını söyledi. Bağıntıda yerine konur:

$$A\,5^n = -4 \cdot \frac{A}{5}\,5^n - 3 \cdot \frac{A}{25}\,5^n + 2 \cdot 5^n$$

5^n sadeleşir ve A'lı terimler sola toplanır (ara adım eklendi):

$$A + \frac{4A}{5} + \frac{3A}{25} = 2 \ \Rightarrow\ \frac{48A}{25} = 2 \ \Rightarrow\ A = \frac{25}{24}$$

Genel çözüm:

$$a_n = C_1\,(-3)^n + C_2\,(-1)^n + \frac{25}{24}\,5^n$$

C_1 ve C_2, $n = 0$ için 6'ya, $n = 1$ için 10'a eşitlenerek bulunur. Hoca bu hesabın uzun sürdüğünü söyleyip yapmadı.

> **Çözüm (derste verilmedi):** $C_1 + C_2 = 6 - \frac{25}{24} = \frac{119}{24}$. $-3C_1 - C_2 = 10 - \frac{125}{24} = \frac{115}{24}$. Toplanınca $-2C_1 = \frac{234}{24}$, $C_1 = -\frac{39}{8}$, $C_2 = \frac{59}{6}$ (Python ile doğrulandı).

8.4 $f(n)$ Üstel, Homojen Çözümde Geçiyor

$$a_n = 2a_{n-1} + 3a_{n-2} + 3^n, \qquad a_0 = 7,\ a_1 = 5$$

Homojen kısım: $r^2 = 2r + 3$, kökler 3 ve -1:

$$a_n^{(h)} = C_1\,3^n + C_2\,(-1)^n$$

Kontrol: $f(n) = 3^n$. Homojen çözümde "tesadüfen" 3^n bir kez geçiyor. Bu yüzden özel çözüm n ile çarpılır. 3^n'in önündeki katsayı 1'dir, ama yazılmadığı için yine bilinmeyen bir A kullanılır:

$$a_n^{(\ddot{o})} = A\,n\,3^n$$

- $a_{n-1} = A(n-1)3^{n-1} = \frac{A}{3}\,n\,3^n - \frac{A}{3}\,3^n$
- $a_{n-2} = A(n-2)3^{n-2} = \frac{A}{9}\,n\,3^n - \frac{2A}{9}\,3^n$

Bağıntıda yerine konur (ara adım eklendi):

$$A\,n\,3^n = 2\left(\frac{A}{3}n3^n - \frac{A}{3}3^n\right) + 3\left(\frac{A}{9}n3^n - \frac{2A}{9}3^n\right) + 3^n$$

$n\,3^n$'li terimler iki tarafta da $A\,n\,3^n$'dir ve birbirini götürür. Geriye kalan 3^n'li terimler:

$$0 = -\frac{2A}{3} - \frac{2A}{3} + 1 \;\Rightarrow\; \frac{4A}{3} = 1 \;\Rightarrow\; A = \frac{3}{4}$$

Genel çözüm:

$$a_n = C_1\,3^n + C_2\,(-1)^n + \frac{3}{4}\,n\,3^n$$

C_1, C_2, $n = 0$ için 7'ye ve $n = 1$ için 5'e eşitlenerek bulunur. Derste hesaplanmadı.

> **Çözüm (derste verilmedi):** $C_1 + C_2 = 7$. $3C_1 - C_2 + \frac{9}{4} = 5 \Rightarrow 3C_1 - C_2 = \frac{11}{4}$. Toplanınca $4C_1 = \frac{39}{4}$, $C_1 = \frac{39}{16}$, $C_2 = \frac{73}{16}$ (Python ile doğrulandı).
>
> **Ders sonrası not:** Slayttaki "polinomun derecesi ne ise a_n'nin de derecesi aynı olur" notu, özel çözüm tahmininin $f(n)$ ile aynı dereceden bir polinom olacağı anlamına gelir. Ancak karakteristik denklemin kökü 1 ise (homojen çözümde 1^n varsa) polinom tahmini de büyük olasılıkla üstel durumdaki gibi n ile çarpılmalıdır; derste bu ayrıntı söylenmedi.

9. Sözel Problemlerden ve Sayı Dizilerinden Rekürans Bağıntısı

Hoca bazen bağıntının matematiksel olarak değil, sözel olarak verileceğini söyledi. O zaman hikâyenin içinden a_n, a_{n-1}, a_{n-2} ilişkisi çıkarılır. Bağıntı kurulduktan sonra çözüm, önceki bölümlerdeki yöntemlerin aynısıdır.

9.1 Dizi 1: $4, 7, 13, 25, 49, 97, \ldots$

Soru: Bu dizide 7., 8., 9. sayılar ne olur? Hoca bu tip soruların eskiden üniversite sınavlarında ve genel yetenek sınavlarında çıktığını söyledi.

Önce ardışık iki terimi toplamayı denedi: $4 + 7 = 11$, olmadı. Sonra her terimi 2 ile çarpıp 1 çıkarmayı denedi:

- $2 \cdot 4 - 1 = 7$
- $2 \cdot 7 - 1 = 13$
- $2 \cdot 13 - 1 = 25$
- $2 \cdot 25 - 1 = 49$

Kural bulundu:

$$a_n = 2a_{n-1} - 1, \qquad a_0 = 4,\ n \geq 1$$

Kontrol: $a_1 = 2 \cdot 4 - 1 = 7$, $a_2 = 2 \cdot 7 - 1 = 13$.

Hoca bağıntıyı bu halde bırakamayacağımızı söyledi. Biri 100. terimi sorarsa a_{99}'u bilmek gerekir. Bağıntının dizi biçimine, yani yalnız n'e bağlı bir ifadeye çevrilmesi gerekir. Bu bağıntı homojen değildir (sabit bir $f(n)$ var). Homojen ve özel kısım ayrılarak çözülür.

9.2 Dizi 2: $1, 3, 7, 17, 41, 99, 239, ...$

Hoca önce ardışık iki terimi toplayıp 2 ile çarpıp 1 çıkarmayı denedi: $(1+3)\cdot 2-1=7$ tuttu, ama $(3+7)\cdot 2-1=19$, olmadı. Sonra ilk iki terimi başlangıç değeri kabul edip "son terimi 2 ile çarp, bir öncekini ekle" kuralını denedi:

- $2\cdot 3+1=7$
- $2\cdot 7+3=17$
- $2\cdot 17+7=41$
- $2\cdot 41+17=99$

$$a_n = 2a_{n-1} + a_{n-2}, \qquad a_0 = 1,\ a_1 = 3,\ n \geq 2$$

Bu ikinci dereceden, sabit katsayılı, homojen bir bağıntıdır. İki başlangıç değeri verilir ve $n \geq 2$ olmalıdır, çünkü hesap 2'den başlar. Yalnız homojen çözümle sonuca ulaşılır.

9.3 Dizi 3: $1, 3, 6, 13, 27, 56, 115, ...$

Hoca bunun bakarak bulunamayacağını söyleyip kuralı verdi:

- $1+3+2^1=6$
- $3+6+2^2=13$
- $6+13+2^3=27$

Son iki terim toplanıp 2'nin bir kuvveti ekleniyor. Kuvvetin 2^n mi, 2^{n-1} mi olacağına karar vermek için a_2 denenir: a_2 hesaplanırken 2^1 eklenmeli, bu yüzden üs $n-1$'dir.

$$a_n = a_{n-1} + a_{n-2} + 2^{n-1}, \qquad a_0 = 1,\ a_1 = 3,\ n \geq 2$$

Kontrol: $a_2 = 3+1+2^1 = 6$. Hoca kontrol sırasında bir an 2^1'i 4 diye hesapladı, sonra düzeltti. Python ile terimler $1, 3, 6, 13, 27, 56, 115$ olarak doğrulandı.

Bu bağıntı homojen değildir. Homojen kısmı karakteristik kökle, 2^{n-1}'li kısmı özel çözümle bulunur. Homojen çözümde 2^{n-1} geçiyorsa özel çözüme n çarpanı eklenir; geçmiyorsa $a_n^{(\ddot{o})} = A\cdot 2^{n-1}$ alınır.

9.4 "00" İçeren 0-1 Dizileri

Problem: Yalnız 0 ve 1'lerden oluşan n birim uzunluğunda diziler oluşturuluyor. n uzunluğunda olup içinde art arda "00" bulunan dizilerin sayısını veren rekürans bağıntısını, başlangıç koşullarıyla birlikte yazın.

Önce olası dizi sayıları: $n=1$ için 2 (0, 1), $n=2$ için 4 (00, 01, 10, 11), $n=3$ için $2^3=8$, $n=4$ için $2^4=16$.

Sayarak ilk değerler:

- $a_0=0$, $a_1=0$: İçinde "00" olamaz.
- $a_2=1$: Yalnız 00.
- $a_3=3$: 000, 001, 100.
- $a_4=8$.

Dizi $0, 0, 1, 3, 8$ diye gider. Hoca, a_{20} sorulsa 2^{20} diziyi tek tek yazıp saymanın mümkün olmadığını, bu yüzden bağıntıya ihtiyaç olduğunu söyledi.

Hoca birkaç deneme yaptı (ilk ikisini toplayıp bir eklemek tutmadı) ve sonunda şu bağıntıyı çıkardı:

$$a_n = 2a_{n-1} + a_{n-2} + 1$$

Kontrol: $a_2 = 2\cdot 0+0+1=1$, $a_3 = 2\cdot 1+0+1=3$, $a_4 = 2\cdot 3+1+1=8$. Hoca bunun homojen olmadığını, homojen kısım ve sabit özel kısım olarak çözüleceğini söyledi.

Ders sonrası not: Hocanın bağıntısı ilk beş değeri tutturuyor, ama sonraki terimlerde yanlıştır. Bu bağıntı $a_5 = 20$ verir. Bütün 5 bitlik diziler Python ile sayıldı: 32 dizinin 19'u "00" içerir. Doğru bağıntı problemin yapısından kurulur: Dizi 1 ile başlıyorsa kalan $n-1$ uzunluk "00" içermelidir (a_{n-1} yol). 01 ile başlıyorsa kalan $n-2$ uzunluk "00" içermelidir (a_{n-2} yol). 00 ile başlıyorsa dizi zaten "00" içerir, kalan $n-2$ basamak serbesttir (2^{n-2} yol). Sonuç: $a_n = a_{n-1} + a_{n-2} + 2^{n-2}$, $n \geq 2$, $a_0 = a_1 = 0$. Terimler: $0, 0, 1, 3, 8, 19, 43, 94, 201$. Birkaç terime uyan bir kural bulmak, bağıntının doğru olduğunu göstermez.

Hoca rekürans bağıntılarının tam bu tip problemleri çözmek için kullanıldığını söyledi.

10. Çözülebilir, Çözülemez ve Çözümsüz Problemler

Dersin sonunda hoca birkaç yazılı slaytı, ileride karşılaşınca ne olduğunu bilelim diye kısaca anlattı.

- **Çözülebilir (tractable):** En kötü durum karmaşıklığı polinom olan bir algoritmayla çözülebilen problem. $\log n$, n^2, n^3 gibi. Hoca polinomun derecesi çok yüksek olursa (n^{100} gibi) problemin çözülebilir olduğunu, ama çok uzun süreceğini söyledi.
- **Çözülemez (intractable):** Polinom zamanlı algoritmayla çözülemeyen problemler. Hoca eldeki donanımın ve imkânların bunlara uygun olmadığını söyledi. Slayta göre endüstride çözülemez diye düşünülen bazı problemlerin, belirli şartlar sağlandığında günlük yaşamda çözülebildiği görülmüştür.
- **Çözümsüz (unsolvable):** Hiçbir algoritmayla çözülemeyen problem. Hocanın örneği Alan Turing'in ortaya attığı durma problemi (halting problem): Girdi olarak başka bir program verildiğinde o program hakkında karar veren bir program yazılması.

Hoca polinom zamanlı ve polinom olmayan karmaşıklıkları da hatırlattı: 2^n ve $n!$ karmaşıklıkları polinom zamanda değildir, "ömrümüz yetmeyebilir". NP-complete ve NP-hard kavramlarını kısaca andı ve ayrıntının Algoritma Analizi dersinde verileceğini söyledi.

Ders sonrası not: Hoca intractable problemleri "polinom zamanlı bir algoritma kullanılsa da çözülemeyen, çözümü hiçbir zaman olmayan" problemler diye anlattı. Doğrusu: Bunlar için polinom zamanlı algoritma bilinmez; mevcut algoritmalar sonlu zamanda çözüm üretir, ama giriş büyüdükçe süre pratikte kabul edilemez hale gelir. Hiçbir algoritmayla çözülemeyen problemler (durma problemi gibi) ayrı bir sınıftır. Durma probleminde sorulan, verilen programın "çözülebilir olup olmadığı" değil, verilen girdiyle durup durmayacağıdır. Turing bunun hiçbir algoritmayla genel olarak çözülemeyeceğini ispatlamıştır; yani "çözülebildiği ispatlanamamış" değil, çözülemediği ispatlanmış bir problemdir.

Ek bilgi (derste anlatılmadı): NP, verilen bir çözümün polinom zamanda doğrulanabildiği problemler sınıfıdır. NP-hard, NP'deki her problemin polinom zamanda kendisine indirgenebildiği problemlerdir. NP-complete ise hem NP'de olan hem NP-hard olan problemlerdir. Derste bu kavramların tanımı belirsiz kaldı.

Örnek Sorular

Soru 1 (vize sorusu, derste çözüldü, 00:14:29): $\sqrt[3]{x^2}$ ifadesinin prefix gösterimini yazın.

Çözüm: $\sqrt[3]{x^2} = x^{2/3}$. Önce $2/3$'ün prefix'i yazılır: $/\,2\,3$. Bu sonuç x'in kuvveti olarak alınır:

$$\uparrow\ x\ /\ 2\ 3$$

Genel olarak $\sqrt[n]{x} \Rightarrow \uparrow\ x\ /\ 1\ n$. "$x(1/n)$" gibi bir yazım olmaz.

Soru 2 (vize sorusu, derste bir öğrenciyle birlikte çözüldü, 01:25:20): $n = 2$, $m = 1$ için postfix ifadenin değerini bulun.

Çözüm (derste yapılan adımlar):

- $n\ 2\ \uparrow$: $n^2 = 4$
- $n\ m\ *$: $n \cdot m = 2$

- $+$: $4 + 2 = 6$
- $3 \uparrow$: $6^3 = 216$
- son çarpma: $216 \cdot 2 = 432$

Sonuç 432'dir. Yanlış yapanların çoğu 532 bulmuştu.

Soru 3 (derste sözlü, 01:27:12): Prefix ve postfix'te üslü ifadeler ters mi yazılır? $\uparrow 3\,2$ ve $3\,2\,\uparrow$ neye eşittir?

Çözüm: Ters yazılmaz; ikisinde de önce taban, sonra üs gelir. $\uparrow 3\,2 = 3^2 = 9$ ve $3\,2\,\uparrow = 3^2 = 9$. Sayıların yeri değişirse ($\uparrow 2\,3$ veya $2\,3\,\uparrow$) sonuç $2^3 = 8$ olur.

Soru 4 (vize sorusu, derste kısmen çözüldü, 00:17:19): Dört girişli (kara yolu, demir yolu, deniz yolu, hava yolu) ve tek çıkışlı $F = \Sigma(4, 8, 9, 10, 12, 14)$ fonksiyonunu Karnaugh haritasıyla sadeleştirin.

Çözüm: Derste bir 4'lük ve iki 2'lik grup olduğu söylendi; ifade yazılmadı.

> **Çözüm (derste verilmedi):** Değişken sırası A, B, C, D (A en anlamlı bit) alınırsa gruplar $\{8, 10, 12, 14\} \Rightarrow A\overline{D}$, $\{8, 9\} \Rightarrow A\overline{B}\,\overline{C}$, $\{4, 12\} \Rightarrow B\overline{C}\,\overline{D}$. Sonuç: $F = A\overline{D} + A\overline{B}\,\overline{C} + B\overline{C}\,\overline{D}$. Python ile bütün seçenekler denendi; bu en kısa ve tek çözümdür.

Soru 5 (vize sorusu, derste çözüldü, 00:19:56): Aşağıdaki kodun karmaşıklığını bulun.

```
for (i = n/2; i <= n; i++)
    for (j = 1; j + n/2 <= n; j++)
        for (k = 1; k <= n; k = k*2)
            ...
```

Çözüm: Dış döngü yaklaşık $n/2$, orta döngü $n/2$, iç döngü $\log_2 n$ kez döner. Toplam $\frac{n^2}{4} \log_2 n$, karmaşıklık $O(n^2 \log n)$. $O(n^3)$ olamaz.

Soru 6 (vize sorusu, derste sözlü, 00:27:20): "Ancak ve ancak finale girerseniz dersi geçebilirsiniz" cümlesini sembolle yazın.

Çözüm: p: "Finale girersiniz", q: "Dersi geçersiniz" denirse $p \leftrightarrow q$. "Ancak ve ancak" XOR değil, çift yönlü oktur.

Soru 7 (derste sözlü, 02:15:05): $4, 7, 13, 25, 49, 97, \ldots$ dizisinde 7., 8. ve 9. sayılar nedir?

Çözüm: Derste kural bulundu: $a_n = 2a_{n-1} - 1$, $a_0 = 4$. Terimlerin değeri söylenmedi.

> **Çözüm (derste verilmedi):** 7. sayı $2 \cdot 97 - 1 = 193$, 8. sayı 385, 9. sayı 769. Bağıntının kapalı çözümü $a_n = 3 \cdot 2^n + 1$'dir (Python ile doğrulandı).

Soru 8 (derste sözlü, 02:26:31): Yalnız 0 ve 1'lerden oluşan n uzunluğundaki dizilerden içinde art arda "00" bulunanların sayısını veren rekürans bağıntısını, başlangıç koşullarıyla yazın.

Çözüm: Derste $a_0 = 0$, $a_1 = 0$, $a_2 = 1$, $a_3 = 3$, $a_4 = 8$ sayıldı ve $a_n = 2a_{n-1} + a_{n-2} + 1$ önerildi. Bu bağıntı $n \geq 5$ için yanlıştır (bkz. Bölüm 9.4).

> **Çözüm (derste verilmedi):** $a_n = a_{n-1} + a_{n-2} + 2^{n-2}$, $n \geq 2$, $a_0 = a_1 = 0$. Terimler $0, 0, 1, 3, 8, 19, 43, 94, 201$; Python ile bütün diziler sayılarak doğrulandı.

Dersin Toparlaması

Ders, vize sorularındaki yaygın hatalarla başladı: Kök içeren ifadelerin prefix'e çevrilirken üs olarak yazılması, Karnaugh haritasının doğru yerleştirilip 1'lerin mümkün olan en büyük gruplarla alınması, yarıya bölünen ya da iki katına çıkan döngülerin $\log n$ karmaşıklığı ve "ancak ve ancak" için $\leftrightarrow$ kullanılması. Ardından özyinelemeli algoritmalar, başlangıç koşulunun gerekliliği, ileri yerine koyma, Gauss toplamı ve faktöriyel üzerinden anlatıldı.

Dersin asıl konusu rekürans bağıntılarıydı. Bir bağıntının içinde kendisinden bir parça bulunur; bu yüzden dizi gibi doğrudan hesaplanmaz ve derecesi kadar başlangıç değeri gerekir. Bağıntılar derece, homojenlik, doğrusallık ve sabit katsayılılığa göre sınıflandırıldı. Sabit katsayılı homojen bağıntılar karakteristik kök yöntemiyle (ayrık ve çakışık kökler), homojen olmayanlar ise homojen çözüm ile özel çözümün toplamı olarak çözüldü. Özel çözümde $f(n)$'nin

türüne (sabit, polinom, üstel) göre tahmin yapılır; üstel terim homojen çözümde geçiyorsa tahmin n ile çarpılır. Sonra sayı dizilerinden ve sözel problemlerden bağıntı kurma örnekleri yapıldı. Ders, çözülebilir, çözülemez ve çözümsüz problemlerin kısa tanıtımıyla bitti. Hoca sonraki derste ağaçlara (trees) geçileceğini, ardından çizgeler (graphs), kromatik polinomlar ve otomatların geleceğini söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.